

```
# Install NLTK
!pip install nltk
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.10/dist-packages (3.8.1)
Requirement already satisfied: click in /usr/local/lib/python3.10/dist-packages (from nltk) (8.1.7)
Requirement already satisfied: joblib in /usr/local/lib/python3.10/dist-packages (from nltk) (1.3.2)
Requirement already satisfied: regex<=2021.8.3 in /usr/local/lib/python3.10/dist-packages (from nltk) (2023.6.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from nltk) (4.66.1)
```

```
!pip install transformers
```

```
Collecting transformers
  Downloading transformers-4.34.1-py3-none-any.whl (7.7 MB)
    7.7/7.7 MB 21.0 MB/s eta 0:00:00
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from transformers) (3.12.4)
Collecting huggingface-hub<1.0,>=0.16.4 (from transformers)
  Downloading huggingface_hub-0.18.0-py3-none-any.whl (301 kB)
    302.0/302.0 kB 20.5 MB/s eta 0:00:00
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (1.23.5)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from transformers) (23.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (6.0.1)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (2023.6.3)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from transformers) (2.31.0)
Collecting tokenizers<0.15,>=0.14 (from transformers)
  Downloading tokenizers-0.14.1-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (3.8 MB)
    3.8/3.8 MB 44.9 MB/s eta 0:00:00
Collecting safetensors>=0.3.1 (from transformers)
  Downloading safetensors-0.4.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (1.3 MB)
    1.3/1.3 MB 41.9 MB/s eta 0:00:00
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.10/dist-packages (from transformers) (4.66.1)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub<1.0,>=0.16.4->trans)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.10/dist-packages (from huggingface-hub<1.0,>=0.16.4)
Collecting huggingface-hub<1.0,>=0.16.4 (from transformers)
  Downloading huggingface_hub-0.17.3-py3-none-any.whl (295 kB)
    295.0/295.0 kB 19.2 MB/s eta 0:00:00
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2.0.7)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2023.7.2)
Installing collected packages: safetensors, huggingface-hub, tokenizers, transformers
Successfully installed huggingface-hub-0.17.3 safetensors-0.4.0 tokenizers-0.14.1 transformers-4.34.1
```

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch
import requests
from bs4 import BeautifulSoup
import re
import nltk
from nltk.corpus import stopwords
```

```
import pandas as pd
```

```
# Read CSV File
df = pd.read_csv('Reports.csv')
```

```
# Extract Column into Tuple
links = tuple(df['links'])
links = tuple(set(links))
```

```
print(links)
print(len(links))
```

```
('https://www.federalreserve.gov/newsevents/pressreleases/monetary20201105a.htm', 'https://www.federalreserve.gov/newsevents/pressr
150
```

```
import re
import pandas as pd
```

```
statement_dates = []
year = []
```

```
for link in links:
    match = re.search(r'\d+', link)
    if match:
        statement_dates.append(match.group())
        year.append(match.group()[4])
```

```
data = {'year': year, 'statements_dates': statement_dates, 'links': links}
```

```

reports = pd.DataFrame(data)

# Convert columns to strings
reports = reports.applymap(str)

# Sort by statement_dates
reports = reports.sort_values(by='statements_dates')

print(reports)

```

	year	statements_dates	links
27	2006	20060131	https://www.federalreserve.gov/newsevents/pres...
51	2006	20060328	https://www.federalreserve.gov/newsevents/pres...
119	2006	20060510	https://www.federalreserve.gov/newsevents/pres...
4	2006	20060629	https://www.federalreserve.gov/newsevents/pres...
136	2006	20060808	https://www.federalreserve.gov/newsevents/pres...
..	...	...	...
125	2023	20230201	https://www.federalreserve.gov/newsevents/pres...
99	2023	20230322	https://www.federalreserve.gov/newsevents/pres...
132	2023	20230503	https://www.federalreserve.gov/newsevents/pres...
110	2023	20230614	https://www.federalreserve.gov/newsevents/pres...
63	2023	20230726	https://www.federalreserve.gov/newsevents/pres...

[150 rows x 3 columns]

```

import requests
from bs4 import BeautifulSoup
import pandas as pd

# Initialize lists to store scraped data
statement_content = []
statement_length = []

for link in links:
    # Send an HTTP GET request to the URL
    response = requests.get(link)

    # Check if the request was successful (status code 200)
    if response.status_code == 200:
        # Parse the HTML content of the page using BeautifulSoup
        soup = BeautifulSoup(response.text, 'html.parser')

        # Find all <p> elements in the HTML
        paragraphs = soup.find_all('p')

        # Filter out irrelevant paragraphs (e.g., preliminary and last paragraphs)
        relevant_paragraphs = paragraphs[1:-1] # Exclude the first and last paragraphs

        # Extract text content from the relevant paragraphs and join them
        statement_text = ' '.join([paragraph.get_text() for paragraph in relevant_paragraphs])

        # Remove line breaks
        statement_text = statement_text.replace('\n', ' ').strip()

        # Append the scraped content and its length to the respective lists
        statement_content.append(statement_text)
        statement_length.append(len(statement_text))

    else:
        print(f"Failed to retrieve content from {link}")

# Create a DataFrame to store the results
data = {'links': links, 'statement_content': statement_content, 'statement_length': statement_length}
reports_df = pd.DataFrame(data)

# Print the DataFrame
print(reports_df)

```

	links \
0	https://www.federalreserve.gov/newsevents/pres...
1	https://www.federalreserve.gov/newsevents/pres...
2	https://www.federalreserve.gov/newsevents/pres...
3	https://www.federalreserve.gov/newsevents/pres...
4	https://www.federalreserve.gov/newsevents/pres...
..	...
145	https://www.federalreserve.gov/newsevents/pres...
146	https://www.federalreserve.gov/newsevents/pres...
147	https://www.federalreserve.gov/newsevents/pres...
148	https://www.federalreserve.gov/newsevents/pres...
149	https://www.federalreserve.gov/newsevents/pres...

		statement_content	statement_length
0	Federal Open Market Committee	Monetary Polic...	4317
1	Federal Open Market Committee	Monetary Polic...	4739
2	Federal Open Market Committee	Monetary Polic...	4610
3	Federal Open Market Committee	Monetary Polic...	5022
4	Federal Open Market Committee	Monetary Polic...	3056
..		...	...
145	Federal Open Market Committee	Monetary Polic...	4881
146	Federal Open Market Committee	Monetary Polic...	4232
147	Federal Open Market Committee	Monetary Polic...	4500
148	Federal Open Market Committee	Monetary Polic...	6721
149	Federal Open Market Committee	Monetary Polic...	3227

[150 rows x 3 columns]

```
dates=reports['statements_dates']
reports_df['statements_dates']=dates
```

```
reports_df
```

	links	statement_content	statement_length
0	https://www.federalreserve.gov/newsevents/pres...	Federal Open Market Committee Monetary Polic...	4317
1	https://www.federalreserve.gov/newsevents/pres...	Federal Open Market Committee Monetary Polic...	4739
2	https://www.federalreserve.gov/newsevents/pres...	Federal Open Market Committee Monetary Polic...	4610
3	https://www.federalreserve.gov/newsevents/pres...	Federal Open Market Committee Monetary Polic...	5022
4	https://www.federalreserve.gov/newsevents/pres...	Federal Open Market Committee Monetary Polic...	3056
...	...	...	...

```
import pandas as pd
```

```
#reports = pd.DataFrame(data)
```

```
# Correct the date where 'statement_dates' is '20070618'
```

```
corrected_date = '20070628'
```

```
reports_df.loc[reports['statements_dates'] == '20070618', 'statements_dates'] = corrected_date
```

```
# Print the updated DataFrame
```

```
print(reports_df)
```

	links \	statement_content	statement_length \
0	https://www.federalreserve.gov/newsevents/pres...	Monetary Polic...	4317
1	https://www.federalreserve.gov/newsevents/pres...	Monetary Polic...	4739
2	https://www.federalreserve.gov/newsevents/pres...	Monetary Polic...	4610
3	https://www.federalreserve.gov/newsevents/pres...	Monetary Polic...	5022
4	https://www.federalreserve.gov/newsevents/pres...	Monetary Polic...	3056
..	...	...	...
145	https://www.federalreserve.gov/newsevents/pres...	Monetary Polic...	4881
146	https://www.federalreserve.gov/newsevents/pres...	Monetary Polic...	4232
147	https://www.federalreserve.gov/newsevents/pres...	Monetary Polic...	4500
148	https://www.federalreserve.gov/newsevents/pres...	Monetary Polic...	6721
149	https://www.federalreserve.gov/newsevents/pres...	Monetary Polic...	3227

	statements_dates
0	20201105
1	20171101
2	20210922

```

3      20160727
4      20060629
..      ...
145     20161214
146     20110126
147     20210127
148     20130918
149     20180926

```

[150 rows x 4 columns]

```

def remove_text_after_voting(text):
    return re.sub(r'Voting for.*', '', text)

# Apply the function to the 'statement_content' column
reports_df['statement_content'] = reports_df['statement_content'].apply(remove_text_after_voting)

# Print the updated DataFrame
print(reports_df)

```

```

                                links \
0  https://www.federalreserve.gov/newsevents/pres...
1  https://www.federalreserve.gov/newsevents/pres...
2  https://www.federalreserve.gov/newsevents/pres...
3  https://www.federalreserve.gov/newsevents/pres...
4  https://www.federalreserve.gov/newsevents/pres...
..      ...
145 https://www.federalreserve.gov/newsevents/pres...
146 https://www.federalreserve.gov/newsevents/pres...
147 https://www.federalreserve.gov/newsevents/pres...
148 https://www.federalreserve.gov/newsevents/pres...
149 https://www.federalreserve.gov/newsevents/pres...

                                statement_content statement_length \
0  Federal Open Market Committee Monetary Polic...          4317
1  Federal Open Market Committee Monetary Polic...          4739
2  Federal Open Market Committee Monetary Polic...          4610
3  Federal Open Market Committee Monetary Polic...          5022
4  Federal Open Market Committee Monetary Polic...          3056
..      ...
145 Federal Open Market Committee Monetary Polic...          4881
146 Federal Open Market Committee Monetary Polic...          4232
147 Federal Open Market Committee Monetary Polic...          4500
148 Federal Open Market Committee Monetary Polic...          6721
149 Federal Open Market Committee Monetary Polic...          3227

```

```

statements_dates
0      20201105
1      20171101
2      20210922
3      20160727
4      20060629
..      ...
145     20161214
146     20110126
147     20210127
148     20130918
149     20180926

```

[150 rows x 4 columns]

```

from transformers import BertForSequenceClassification, BertTokenizer

finbert = BertForSequenceClassification.from_pretrained('ahmedrachid/FinancialBERT-Sentiment-Analysis', num_labels=3)
tokenizer = BertTokenizer.from_pretrained('ahmedrachid/FinancialBERT-Sentiment-Analysis')

```

```

Downloading (...)lve/main/config.json: 789/789 [00:00<00:00,
100% 23.0kB/s]

Downloading pytorch_model.bin: 439M/439M [00:04<00:00,
100% 115MB/s]

Downloading 369/369 [00:00<00:00,
(...)okenizer_config.json: 100% 16.8kB/s]

Downloading 226k/226k [00:00<00:00,

```

```

import requests
from bs4 import BeautifulSoup

```

```

from transformers import pipeline
from transformers import BertTokenizer
import nltk
from nltk.corpus import stopwords

# Step 1: Load the finbert-tone model and set up tokenizer
model = BertForSequenceClassification.from_pretrained('ahmedrachid/FinancialBERT-Sentiment-Analysis', num_labels=3)
tokenizer = BertTokenizer.from_pretrained('ahmedrachid/FinancialBERT-Sentiment-Analysis')
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
label_map = {0: "neutral", 1: "positive", 2: "negative"}

# Step 2: Define function for sentiment analysis
def analyze_sentiment(statement):
    inputs = tokenizer(statement, return_tensors="pt")
    inputs.to(device)

    with torch.no_grad():
        outputs = model(**inputs)

    predicted_label = torch.argmax(outputs.logits).item()
    predicted_sentiment = label_map[predicted_label]

    return predicted_sentiment

# Step 3: Define function to process statements
def process_statements(row):
    lines = row['statement_content'].split('.')
    scores = [analyze_sentiment(line) for line in lines]

    positive_scores = (scores.count('positive') / len(scores))*100
    negative_scores = (scores.count('negative') / len(scores))*100
    neutral_scores = (scores.count('neutral') / len(scores))*100
    if(positive_scores>negative_scores and positive_scores>neutral_scores):
        tone='positive'
    elif(positive_scores > negative_scores and positive_scores>=neutral_scores):
        tone='neutral'
    elif(positive_scores < negative_scores and negative_scores>=neutral_scores):
        tone='negative'
    elif(neutral_scores > negative_scores and positive_scores<neutral_scores):
        tone='neautral'

    return pd.Series({
        'Year': row['statements_dates'][:4],
        'Date': pd.to_datetime(row['statements_dates'], format='%Y%m%d').strftime('%d-%m-%Y'),
        'Statement': row['statement_content'],
        'Mean Positive': positive_scores,
        'Mean Negative': negative_scores,
        'Mean Neutral': neutral_scores,
        'Tone' : tone
    })

# Define a function to find the overall sentiment score for a year
def overall_sentiment(year_data):
    max_mean = max(year_data['Mean Positive'], year_data['Mean Negative'], year_data['Mean Neutral'])
    if max_mean == year_data['Mean Positive']:
        return 'positive'
    elif max_mean == year_data['Mean Negative']:
        return 'negative'
    else:
        return 'neutral'

# Step 4: Apply the processing function to the reports_df DataFrame
results_df = reports_df.apply(process_statements, axis=1)
results_df.to_excel('fomc_final_sheets.xlsx', index=False)

# Create a new DataFrame for yearly scores
yearly_scores = results_df.groupby('Year').agg({'Mean Positive': 'max', 'Mean Negative': 'max', 'Mean Neutral': 'max'})
yearly_scores['Overall Score'] = yearly_scores.apply(overall_sentiment, axis=1)

# Create an Excel writer
with pd.ExcelWriter('fomc_final_sheets.xlsx', engine='openpyxl', mode='a') as writer:
    # Write the results_df to the Excel file
    #results_df.to_excel(writer, sheet_name='Results', index=False)

    # Write the yearly_scores to the Excel file in a new sheet
    yearly_scores.reset_index(inplace=True)

```

```
yearly_scores.to_excel(writer, sheet_name='Yearly Scores', index=False)
```

```
import pandas as pd
```

```
# Load the Excel file
```

```
df = pd.read_excel('fomc_final_sheets.xlsx')
```

```
# Now 'df' contains the data from the Excel file
```

```
print(df)
```

	Year	Date		Statement \
0	2020	05-11-2020	Federal Open Market Committee	Monetary Polic...
1	2017	01-11-2017	Federal Open Market Committee	Monetary Polic...
2	2021	22-09-2021	Federal Open Market Committee	Monetary Polic...
3	2016	27-07-2016	Federal Open Market Committee	Monetary Polic...
4	2006	29-06-2006	Federal Open Market Committee	Monetary Polic...
..	...	...		...
145	2016	14-12-2016	Federal Open Market Committee	Monetary Polic...
146	2011	26-01-2011	Federal Open Market Committee	Monetary Polic...
147	2021	27-01-2021	Federal Open Market Committee	Monetary Polic...
148	2013	18-09-2013	Federal Open Market Committee	Monetary Polic...
149	2018	26-09-2018	Federal Open Market Committee	Monetary Polic...

	Mean Positive	Mean Negative	Mean Neutral	Tone
0	63.636364	22.727273	13.636364	positive
1	50.000000	31.818182	18.181818	positive
2	64.000000	32.000000	4.000000	positive
3	47.826087	43.478261	8.695652	positive
4	55.555556	11.111111	33.333333	positive
..	...	...	...	...
145	50.000000	50.000000	0.000000	positive
146	46.666667	26.666667	26.666667	positive
147	60.869565	26.086957	13.043478	positive
148	33.333333	57.142857	9.523810	negative
149	57.142857	42.857143	0.000000	positive

```
[150 rows x 7 columns]
```